RecSysX — Presentation Guide

Use this as a 5–7 minute speaking guide with a live demo.

Opening — 20 seconds

RecSysX is a hybrid recommendation engine built to demonstrate both machine-learning reasoning and software-engineering delivery. The demo is local and reproducible, so I can explain every step without depending on a paid service.

Problem — 40 seconds

A useful recommender must personalize results even when behavioral data is sparse. RecSysX combines content similarity, collaborative user behavior and popularity while exposing score components for debugging and explainability.

Architecture — 60 seconds

Draw or explain the pipeline from input/data source to model/retrieval logic, then to the FastAPI service and dashboard. Emphasize the separation between model logic and product/API logic.

Live Demo — 2 to 3 minutes

Follow the numbered demo flow below. Pause at the algorithmic output and explain what the model/system is doing rather than clicking quickly through screens.

- Step 1: Select user u1 and show interaction history.
- Step 2: Explain why already-seen items are removed.
- Step 3: Walk through content and collaborative score components for a recommendation.
- Step 4: Switch users and show personalized changes.
- Step 5: Explain scale-up using learned two-tower embeddings plus an ANN index such as FAISS.

Engineering — 45 seconds

Mention validation, tests, Docker, CI and the reason the demo can run locally. Then identify the optional components that would be used in a larger deployment.

Close — 30 seconds

Summarize one technical result, one design trade-off and one next improvement. Invite questions about model choice, metrics, scale or failure handling.

Resume-ready bullets

- Built a hybrid recommendation engine combining content embeddings, collaborative user behavior and popularity signals to generate personalized top-K product recommendations.
- Implemented score explainability, cold-start-friendly content features, offline catalog-coverage metrics, FastAPI model serving, Docker deployment and automated recommendation tests.
- Added optional PyTorch two-tower embedding and FAISS ANN modules for scaling the same API toward learned retrieval and low-latency vector search.

Metrics to mention if asked

- Precision@K
- Recall@K
- NDCG@K
- Hit Rate@K
- Catalog coverage / diversity

30-second fallback if the live demo fails

> *“The application normally runs locally through FastAPI. I also keep a saved deterministic output and automated tests in the repository. I will show the expected output first, then walk through the code path that produces it and the test that verifies the behavior.”*

Rapid Interview Q&A;

How do you evaluate?

Precision@K, Recall@K, NDCG@K, hit rate, coverage, diversity, novelty and online CTR/conversion.

How would you serve millions of items?

Learn user/item embeddings, precompute item vectors, use FAISS/HNSW ANN retrieval, then apply a smaller ranking model to a few hundred candidates.

Before the interview

- Run the app once from a clean terminal.
- Run `pytest -q` and confirm all tests pass.
- Know the main API endpoint and one representative request/response.
- Know one limitation and one scale-up plan.
- Do not claim optional technology as trained/deployed unless you have actually run it.